Course Name and Code:	**Structured Programming Methodology LAB (316U06C107)**	**Semester:**	**FY SEM I**
Date of Performance:	**12 / 10 / 2026**	**Div and Batch No:**	**F-1**
Faculty Name:	**Prof. Jayesh Rane**	**Roll No:**	**16010426069**
Faculty Sign & Date:		**Grade/Marks:**	**___/15**

Assignment No: 1

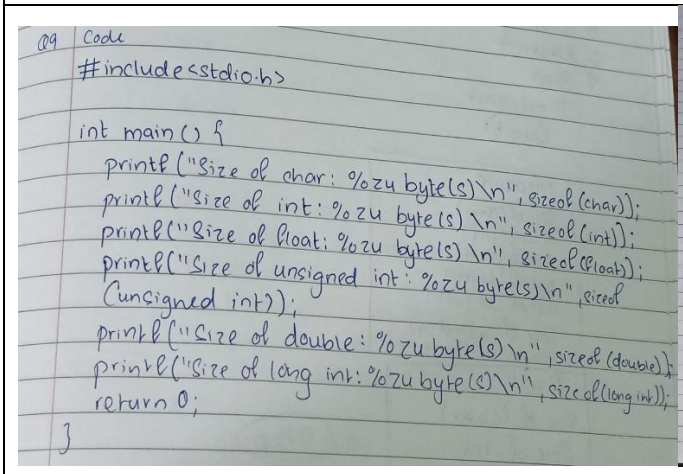
Note:

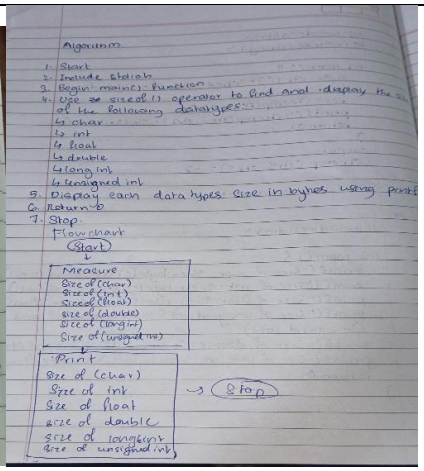
1. Mention the Q. No. and the statement of the program allotted to you as per your roll number.
2. Write the complete C program for each assigned question and include the corresponding output.
3. Attach scanned copy of **handwritten algorithm and/or flowchart for each program.**

Section A — Data Types & Operators

Q. No:9. Write a program that uses the sizeof operator to display the number of bytes occupied by each fundamental C data type (int, float, double, char), and at least one qualified type (e.g., long int or unsigned int).

Code:





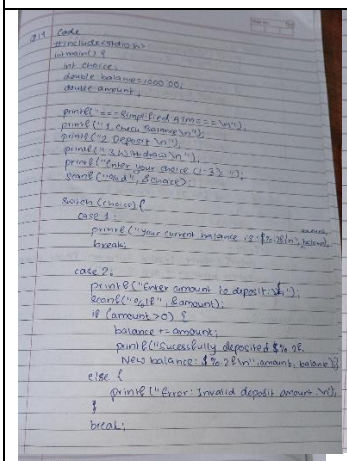
Output:

```
"C:\Users\Student\Downloads
Size of char: 1 byte(s)
Size of int: 4 byte(s)
Size of float: 4 byte(s)
Size of double: 8 byte(s)
Size of long int: 4 byte(s)
Size of unsigned int: 4 byte(s)
```

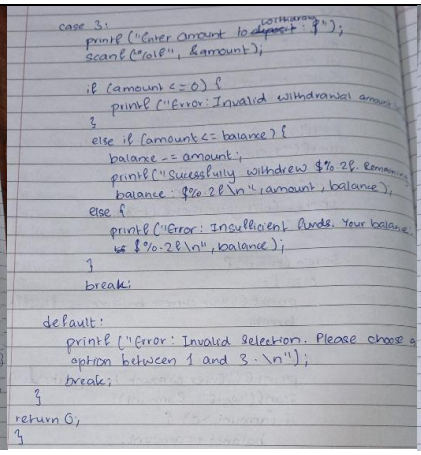
Section B — Decision Making & Branching

Q. No: 19. Write a menu-driven program (using switch-case) for a simplified ATM that lets the user select one of: Check Balance, Deposit, or Withdraw, for a single transaction. Assume a fixed starting balance. For a withdrawal, use an if-else check to ensure the amount does not exceed the balance.

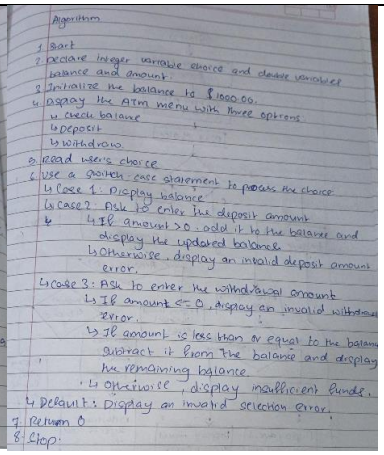
Code:



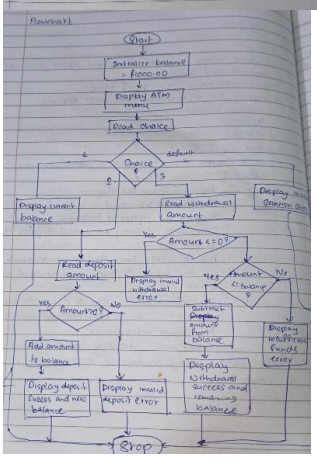
```
#include<stdio.h>
int main() {
    int choice;
    double balance=1000.00;
    double amount;
    printf("==Simplified ATM==\n");
    printf("1.Check Balance\n");
    printf("2.Deposit\n");
    printf("3.Withdraw\n");
    printf("Enter your choice (1-3): ");
    scanf("%d", &choice);
    switch(choice) {
    case 1:
        printf("Your current balance is: $%.2f\n", balance);
        break;
    case 2:
        printf("Enter amount to deposit: $");
        scanf("%lf", &amount);
        if (amount>0) {
            balance += amount;
            printf("Successfully deposited $%.2f.
            New balance: $%.2f\n", amount, balance);
        } else {
            printf("Error: Invalid deposit amount.\n");
        }
        break;
```



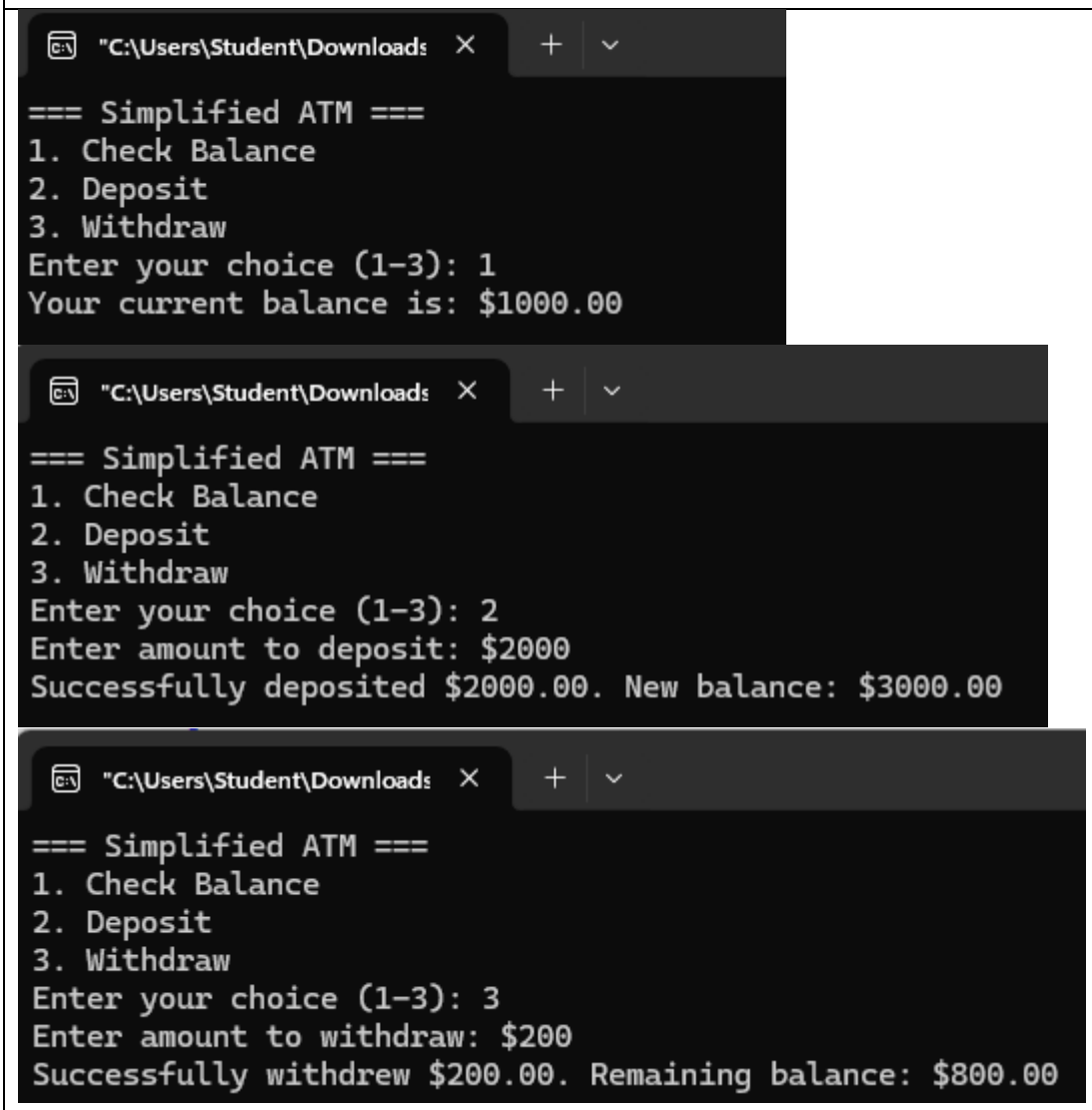
```
    case 3:
        printf("Enter amount to withdraw: $");
        scanf("%lf", &amount);
        if (amount <= 0) {
            printf("Error: Invalid withdrawal amount.\n");
        }
        else if (amount <= balance) {
            balance -= amount;
            printf("Successfully withdrew $%.2f. Remaining
            balance: $%.2f\n", amount, balance);
        else {
            printf("Error: Insufficient funds. Your balance
            is $%.2f\n", balance);
        }
        break;
    default:
        printf("Error: Invalid selection. Please choose a
        option between 1 and 3.\n");
        break;
    }
    return 0;
}
```



```
Algorithm
1. Start
2. Declare integer variable choice and double variables
   balance and amount.
3. Initialize the balance to $1000.00.
4. Display the ATM menu with three options:
   1. Check balance
   2. Deposit
   3. Withdraw.
5. Read user's choice.
6. Use a switch-case statement to process the choice:
   - Case 1: Display balance.
   - Case 2: Ask to enter the deposit amount.
     - If amount > 0, add it to the balance and
       display the updated balance.
     - Otherwise, display an invalid deposit amount
       error.
   - Case 3: Ask to enter the withdrawal amount.
     - If amount <= 0, display an invalid withdrawal
       error.
     - If amount is less than or equal to the balance,
       subtract it from the balance and display
       the remaining balance.
     - Otherwise, display insufficient funds.
   - Default: Display an invalid selection error.
7. Return 0.
8. Stop.
```



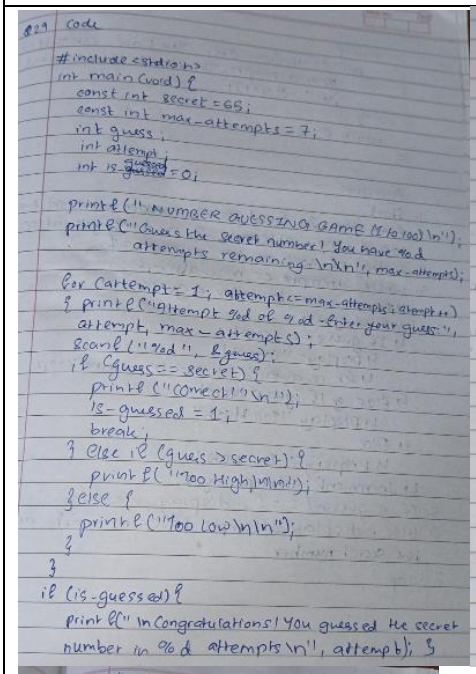
Output:


"C:\Users\Student\Downloads
=== Simplified ATM ===
1. Check Balance
2. Deposit
3. Withdraw
Enter your choice (1-3): 1
Your current balance is: $1000.00
"C:\Users\Student\Downloads
=== Simplified ATM ===
1. Check Balance
2. Deposit
3. Withdraw
Enter your choice (1-3): 2
Enter amount to deposit: $2000
Successfully deposited $2000.00. New balance: $3000.00
"C:\Users\Student\Downloads
=== Simplified ATM ===
1. Check Balance
2. Deposit
3. Withdraw
Enter your choice (1-3): 3
Enter amount to withdraw: $200
Successfully withdrew $200.00. Remaining balance: $800.00

Section C — Looping Control Structures

Q. No: 29. Write a number-guessing game: the program stores a fixed secret number between 1 and 100, and the user has 7 attempts to guess it. After each guess, display "Too High", "Too Low", or "Correct!", using a loop that ends when the number is guessed or the attempts run out.

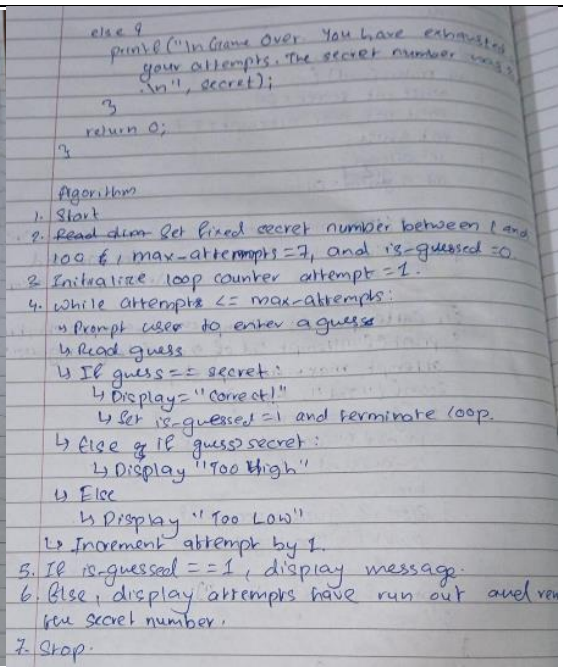
Code:



```
#include <stdio.h>
int main(void) {
    const int secret = 65;
    const int max_attempts = 7;
    int guess;
    int attempt;
    int is_guessed = 0;

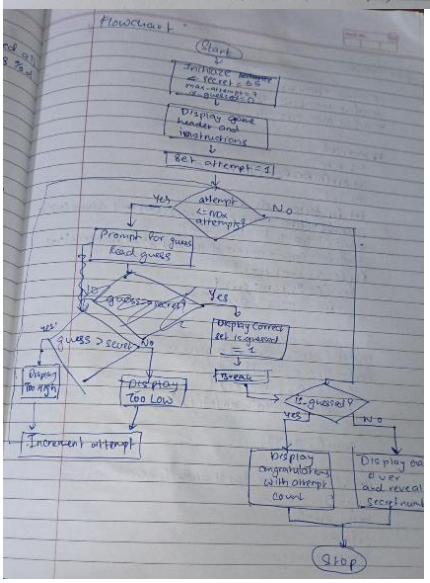
    printf("NUMBER GUESSING GAME (1 to 100)\n");
    printf("Guess the secret number! You have %d
            attempts remaining.\n\n", max_attempts);

    for (attempt = 1; attempt <= max_attempts; attempt++) {
        printf("Attempt %d of %d - Enter your guess: ",
               attempt, max_attempts);
        scanf("%d", &guess);
        if (guess == secret) {
            printf("Correct!\n\n");
            is_guessed = 1;
            break;
        } else if (guess > secret) {
            printf("Too High\n\n");
        } else {
            printf("Too Low\n\n");
        }
    }
    if (is_guessed) {
        printf("Congratulations! You guessed the secret
                number in %d attempts\n", attempt);
    } else {
        printf("Game Over. You have exhausted
                your attempts. The secret number was
                %d\n", secret);
    }
    return 0;
}
```



Algorithm

1. Start
2. Read (or) Set fixed secret number between 1 and 100 & max_attempts = 7, and is_guessed = 0
3. Initialize loop counter attempt = 1.
4. While attempt <= max_attempts:
   - Prompt user to enter a guess
   - Read guess
   - If guess == secret:
     - Display = "Correct!"
     - Set is_guessed = 1 and terminate loop.
   - Else if guess > secret:
     - Display "Too High"
   - Else
     - Display "Too Low"
   - Increment attempt by 1.
5. If is_guessed == 1, display message.
6. Else, display attempts have run out and reveal the secret number.
7. Stop.



Output:

```
Output

==================================================
        NUMBER GUESSING GAME (1 to 100)
==================================================
Guess the secret number! You have 7 attempts.

Attempt 1 of 7 - Enter your guess: 50
Too Low

Attempt 2 of 7 - Enter your guess: 70
Too High

Attempt 3 of 7 - Enter your guess: 60
Too Low

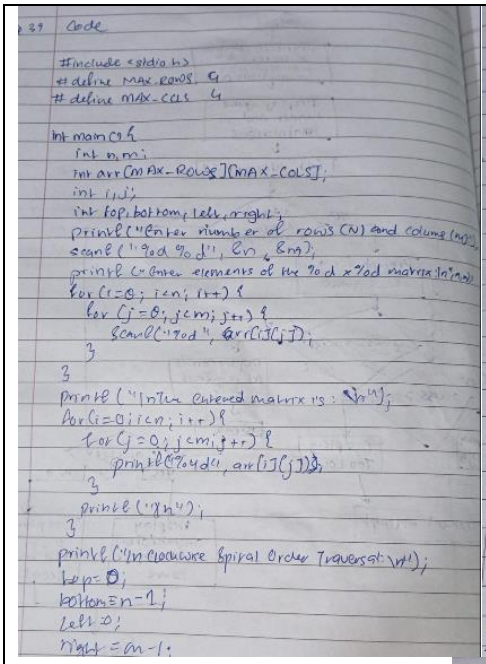
Attempt 4 of 7 - Enter your guess: 65
Correct!

Congratulations! You guessed the secret number in 4 attempt(s)!
```

Section D — Arrays

Q. No: 39. Write a program to accept an N×M 2D array and display its elements in clockwise spiral order.

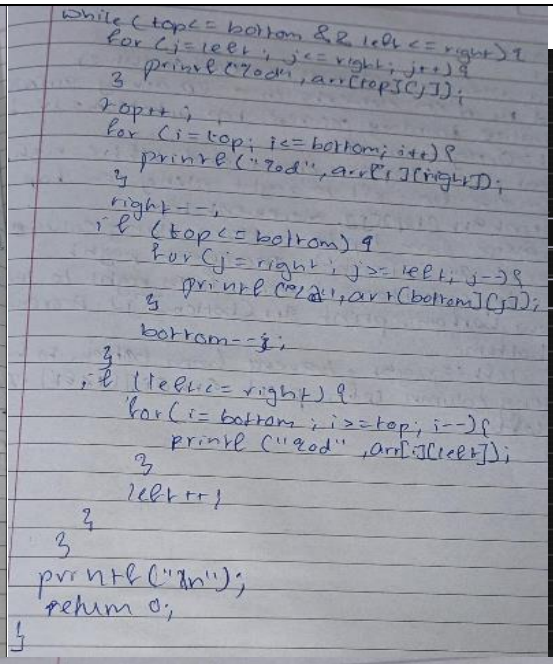
Code:



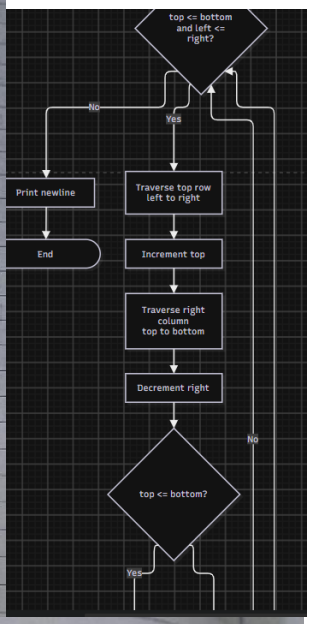
```
Code

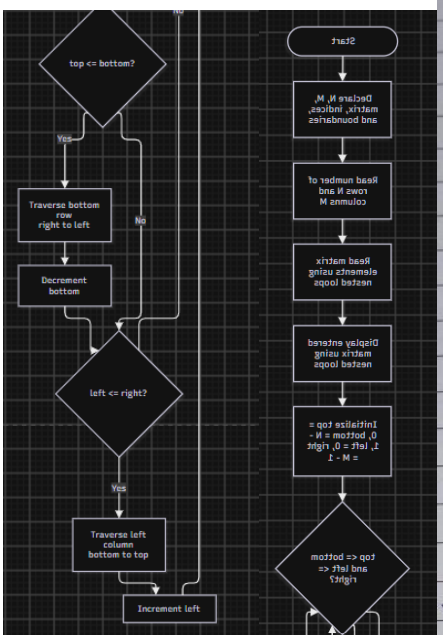
#include <stdio.h>
#define MAX_ROWS 6
#define MAX_COLS 6

int main() {
    int n, m;
    int arr[MAX_ROWS][MAX_COLS];
    int i, j;
    int top, bottom, left, right;
    printf("Enter number of rows (N) and columns (M): ");
    scanf("%d %d", &n, &m);
    printf("Enter elements of the %d x %d matrix:\n", n, m);
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &arr[i][j]);
        }
    }
    printf("\nThe entered matrix is:\n");
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            printf("%4d", arr[i][j]);
        }
        printf("\n");
    }
    printf("\nClockwise Spiral Order Traversal:\n");
    top=0;
    bottom=n-1;
    left=0;
    right=m-1;
```



```
while(top<=bottom && left<=right) {
    for(j=left; j<=right; j++) {
        printf("%4d", arr[top][j]);
    }
    top++;
    for(i=top; i<=bottom; i++) {
        printf("%4d", arr[i][right]);
    }
    right--;
    if(top<=bottom) {
        for(j=right; j>=left; j--) {
            printf("%4d", arr[bottom][j]);
        }
        bottom--;
    }
    if(left<=right) {
        for(i=bottom; i>=top; i--) {
            printf("%4d", arr[i][left]);
        }
        left++;
    }
}
printf("\n");
return 0;
}
```





Algorithm

1. Start
2. Read dimensions N (rows) and M (columns)
3. Read the N x m elements in a 2D array arr[N][M]
4. Initialize boundary pointers: top=0, bottom=N-1, left=0, right=m-1.
5. Repeat while: top<=bottom and left<=right
   - Traverse from left to right along row top, print arr[top][j], increment top.
   - Traverse from top to bottom along column right, print arr[i][right], Decrement right.
   - If top<=bottom, traverse from right to left along row bottom, print arr[bottom][j], Decrement bottom.
   - If left<=right, traverse from bottom to top along column left, print arr[i][left]. Increment left.
6. Stop.

Output:

```
Output

Enter number of rows (N) and columns (M): 2
2
Enter elements of the 2 x 2 matrix:
9
3
2
4

The entered matrix is:
   9    3
   2    4

Clockwise Spiral Order Traversal:
9 3 4 2
```

Section E — Strings & String Handling Functions

Q. No: 49. Write a program to check whether a given identifier name follows valid C naming rules: it must start with a letter or underscore, contain only letters/digits/ under-scores, and must not match any name in a small fixed list of C reserved keywords.

Code:

main.c — Run ▶

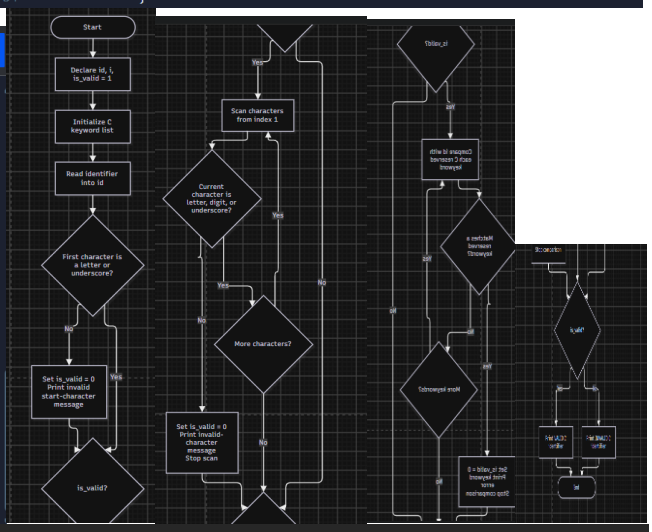
```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char id[100];
    int i;
    int is_valid = 1;
    const char *keywords[] = {
        "auto", "break", "case", "char", "const", "continue",
"default", "do",
        "double", "else", "enum", "extern", "float", "for", "goto",
"if",
        "int", "long", "register", "return", "short", "signed",
"sizeof", "static",
        "struct", "switch", "typedef", "union", "unsigned", "void",
"volatile", "while"
    };
    int num_keywords = sizeof(keywords) / sizeof(keywords[0]);
    printf("Enter an identifier name to validate: ");
```

main.c — Run ▶

```c
    scanf("%99s", id);
    if (!((id[0] >= 'a' && id[0] <= 'z') ||
          (id[0] >= 'A' && id[0] <= 'Z') ||
          id[0] == '_')) {
        is_valid = 0;
        printf("Invalid: Identifier must start with a letter or an
underscore.\n");
    }
    if (is_valid) {
        for (i = 1; id[i] != '\0'; i++) {
            if (!((id[i] >= 'a' && id[i] <= 'z') ||
                  (id[i] >= 'A' && id[i] <= 'Z') ||
                  (id[i] >= '0' && id[i] <= '9') ||
                  id[i] == '_')) {
                is_valid = 0;
                printf("Invalid: Character '%c' is not allowed (only
letters, digits, and underscores).\n", id[i]);
                break;
            }
```

main.c — Run ▶

```c
            }
        }
    }
    if (is_valid) {
        for (i = 0; i < num_keywords; i++) {
            if (strcmp(id, keywords[i]) == 0) {
                is_valid = 0;
                printf("Invalid: '%s' is a C reserved keyword.\n", id);
                break;
            }
        }
    }
    if (is_valid) {
        printf("Result: '%s' is a VALID C identifier.\n", id);
    } else {
        printf("Result: '%s' is an INVALID C identifier.\n", id);
    }
    return 0;
}
```



Algorithm
1.Start
2.Define a fixed list of C reserved keywords (e.g., auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while).
3.Read the identifier string from the user.
4.Initialize flag is_valid = 1.
5.Check Rule 1 (First Character):
6.If the first character is not an uppercase letter (A-Z), lowercase letter (a-z), or underscore (_), set is_valid = 0.
7.Check Rule 2 (Subsequent Characters):
8.If is_valid == 1, loop through all remaining characters until \0.
9.If any character is not a letter, digit (0-9), or underscore (_), set is_valid = 0 and break.
10.Check Rule 3 (Reserved Keyword Check):
11.If is_valid == 1, compare the input string against each keyword in the keyword list using strcmp().
12.If a match is found, set is_valid = 0 and break.
13.If is_valid == 1, display that the identifier is valid; otherwise, display that it is invalid with the reason.
14.Stop

Output:

Output

```
Enter an identifier name to validate: int
Invalid: 'int' is a C reserved keyword.
Result: 'int' is an INVALID C identifier.
```

Signature of faculty in-charge with Date: